

Anti-Phishing in the Era of Deepfakes: Deepfake Audio Detection for Voice Impersonation

Ma Ka Yu

Student ID: 730055167

Abstract

This project investigates deepfake audio detection as a defence against phishing, vishing, and voice impersonation attacks. As modern text-to-speech and voice synthesis tools become increasingly accessible, attackers can generate convincing synthetic speech that imitates trusted individuals and bypasses traditional phishing indicators such as suspicious links or written language. To address this threat, the project designs, implements, and evaluates a machine learning system for classifying speech recordings as real or fake.

A labelled dataset was constructed from multiple genuine and synthetic audio sources, including LJSpeech, Common Voice, VCTK, WaveFake, ASVspoof 2019, and Kokoro-82M generated samples. Audio files were standardised through resampling, fixed-duration processing, log-Mel spectrogram extraction, and training-set normalisation. A hybrid CNN-BiLSTM model was developed to capture both local spectral artefacts and temporal speech patterns. The system was evaluated using accuracy, precision, recall, F1-score, confusion matrix analysis, ROC-AUC, precision-recall analysis, threshold tuning, inference time, and external generalisation testing.

The final model achieved strong performance on the held-out test set, demonstrating that spectrogram-based deep learning can identify many forms of synthetic speech effectively. However, testing on externally generated audio also showed limitations in generalising to unseen synthesis tools. The project therefore contributes both a functional prototype detector and a critical assessment of its practical strengths and weaknesses. Overall, the work demonstrates the potential of automated deepfake audio screening while highlighting the need for broader datasets, improved generalisation, and integration with verification workflows for real-world anti-phishing use.

☒ I certify that all material in this dissertation which is not my own work has been identified.

Signature: _____ Ma Ka Yu

Contents

1	Introduction	1
1.1	Background and Context	1
1.2	Problem and Motivation	1
1.3	Aims and Objectives	1
2	Design and Methodology	2
2.1	Overall System Architecture	2
2.2	Dataset Construction	3
2.3	Feature Extraction	4
2.4	Model Design	5
2.5	Training Strategy	6
2.6	Prototype System	7
3	Implementation	8
3.1	Development Environment	8
3.2	Data Loading and Preprocessing	8
3.3	Feature Extraction and Normalisation	8
3.4	Model Implementation	9
3.5	Training Implementation	9
3.6	Prototype Inference Implementation	9
3.7	Iterative Development	10
3.7.1	Version 1: Initial Implementation	10
3.7.2	Version 2: Normalisation Fix and Dataset Expansion	10
3.7.3	Version 3: Final Pipeline	11
3.8	Implementation Challenges	11
4	Results and Evaluation	11
4.1	Evaluation Overview	11
4.2	Full Test Set Performance	12
4.3	Confusion Matrix Analysis	12
4.4	Visual Evaluation on Balanced Subset	13
4.4.1	ROC Curve	13
4.4.2	Precision-Recall Curve	14
4.5	Threshold Tuning	14
4.6	Per-Class Performance on the Subset	15
4.7	Dataset and Source Analysis	15
4.8	Inference Time Evaluation	16
4.9	External Generalisation Test	16
4.10	Overall Evaluation	17
5	Discussion and Reflection	17
5.1	Discussion	17
5.2	Limitations	18
5.3	Reflection	18
5.4	Evaluation Against Objectives	19
5.5	Future Work	19
6	Conclusion	20
A	Appendix	24

A.1	Generative AI Prompt Log	24
-----	------------------------------------	----

1 Introduction

1.1 Background and Context

Recent advances in artificial intelligence have made it increasingly easy to generate realistic synthetic media, commonly known as deepfakes [1][2]. In the audio domain, modern text-to-speech and voice synthesis systems can produce speech that closely resembles genuine human recordings. Although these technologies have legitimate uses in accessibility, media production, education, and entertainment, they also create new opportunities for misuse when applied to impersonation and social engineering.

One particularly concerning application is deepfake-enabled phishing, especially voice phishing, or vishing.[3][4] Traditional phishing attacks often rely on written messages, malicious links, or fraudulent websites[3], which can sometimes be detected through suspicious wording, formatting errors, or known security filters. Audio-based phishing is more difficult to assess because it exploits trust in a speaker’s voice rather than visible textual indicators. If an attacker can imitate a trusted individual, such as a colleague, manager, family member, or organisation representative, the victim may be more likely to comply with urgent or sensitive requests[4][5].

This creates a challenge for both users and existing cybersecurity tools. Conventional anti-phishing systems are mainly designed for emails, URLs, and message content, and are therefore not sufficient for detecting synthetic speech. At the same time, human listeners may struggle to reliably distinguish high-quality generated speech from genuine recordings[6], particularly under pressure or in realistic communication scenarios. As a result, automated audio analysis has become an important direction for improving defences against deepfake-assisted phishing.

1.2 Problem and Motivation

The increasing accessibility of deepfake generation tools has lowered the barrier for attackers to conduct sophisticated impersonation attacks. In particular, voice cloning and text-to-speech systems can now replicate speech patterns with limited input data[6], enabling convincing audio-based phishing attempts. This presents a critical challenge for cybersecurity, as victims of voice phishing may have no visual evidence to inspect and may rely primarily on whether a voice sounds familiar or trustworthy.

Current anti-phishing solutions are largely designed to detect traditional threats, such as malicious links, suspicious emails, or fraudulent websites. These approaches are insufficient when facing audio-based deception, where the attack relies on trust in the speaker rather than textual indicators. Phone scams and voice impersonation attacks are especially relevant because they can occur in real time, under pressure, and without the visual cues that might be available in video-based communication [4].

The motivation for this project is therefore to address this gap by developing a machine learning-based approach for detecting deepfake audio in phishing and voice impersonation scenarios. Although deepfake phishing can involve both audio and video, this project focuses on audio because voice-based attacks are highly practical, difficult for users to verify during live communication, and increasingly accessible through modern speech synthesis tools. This focus also provides a foundation for future synthetic audio detection tasks, including unauthorised voice cloning and AI-generated speech or singing.

Given the high-risk nature of phishing attacks, the system should not only achieve strong classification performance but also support practical use. This includes consistent preprocessing, efficient inference, and awareness of the trade-off between detecting synthetic speech and avoiding excessive false alarms.

1.3 Aims and Objectives

The primary aim of this project is to design, implement, and evaluate an artificial intelligence-based system capable of detecting deepfake audio that may be used in phishing, vishing, or voice impersonation attacks.

To achieve this aim, the project is guided by the following objectives:

- Construct a labelled dataset containing both genuine and synthetic speech samples from multiple sources.
- Apply appropriate preprocessing and feature extraction techniques to convert audio recordings into machine learning-ready representations.
- Develop and train a deep learning model capable of classifying audio samples as real or synthetic.
- Evaluate the model using standard classification metrics, including accuracy, precision, recall, F1-score, ROC-AUC, precision-recall analysis, and confusion matrix analysis.
- Analyse practical system behaviour, including threshold selection, inference time, and limitations when tested on unseen or externally generated audio.
- Implement a prototype inference workflow that demonstrates how the trained detector could support real-time or near-real-time deepfake audio screening.

By addressing both model performance and practical usability, this project contributes towards the development of automated defences against deepfake-assisted voice phishing and synthetic audio impersonation.

2 Design and Methodology

2.1 Overall System Architecture

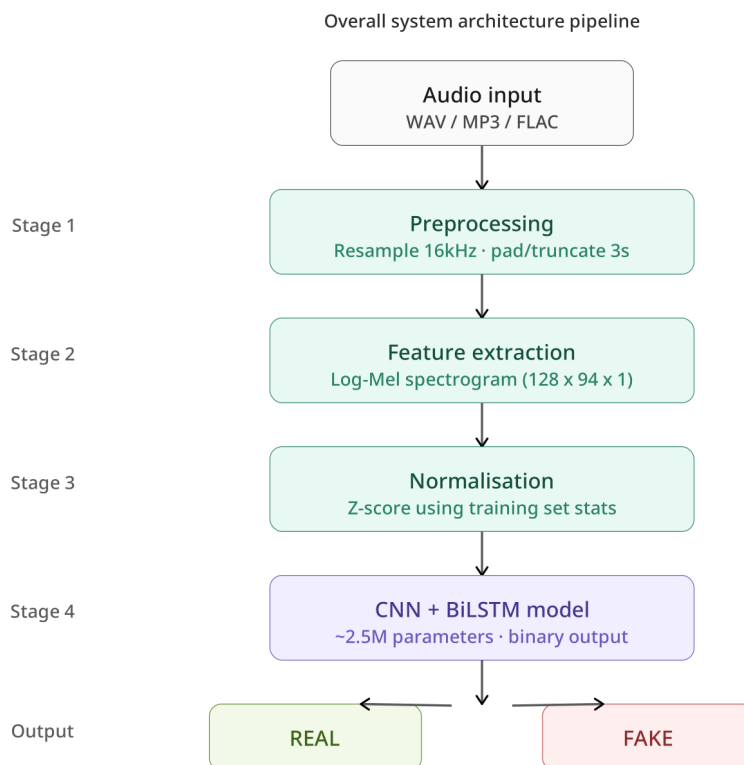


Figure 1: Overall system architecture for the proposed deepfake audio detection pipeline.

The proposed system follows a structured pipeline for detecting deepfake audio in phishing and voice impersonation scenarios. The design transforms raw audio files into a binary classification output while maintaining sufficient computational efficiency for near-real-time use.

The system consists of four main stages: dataset construction, feature extraction, model classification, and prototype inference. In the first stage, genuine and synthetic speech samples are collected from multiple sources to create a labelled dataset. These audio files are then standardised and converted into log-Mel spectrograms, which provide a time-frequency representation suitable for deep learning. The extracted features are passed into a CNN-BiLSTM classifier, which produces a probability score indicating whether the input audio is likely to be synthetic. This probability is then converted into a final classification label of *REAL* or *FAKE*.

A modular architecture was adopted so that each stage of the pipeline could be developed and evaluated independently. This is important for deepfake audio detection because both generative models and detection methods are rapidly evolving. For example, the dataset construction process can be expanded with new synthesis sources without redesigning the model, while the feature extraction stage can be adjusted if more effective audio representations are identified in future work.

The system was also designed with practical deployment constraints in mind. Since a phishing defence tool would need to process incoming audio quickly, the model architecture was selected to balance classification performance with inference speed. Although the current system is a proof of concept rather than a fully deployed real-time application, its inference workflow demonstrates how deepfake audio detection could be integrated into a user-facing cybersecurity tool.

2.2 Dataset Construction

A large and diverse dataset was constructed to train and evaluate the deepfake audio detector. The final dataset contained both genuine and synthetic speech samples, with genuine audio sourced primarily from LJSpeech[7] and Common Voice[8], and synthetic audio sourced from WaveFake[9], ASVspoof2019 Logical Access (LA)[10][11], and Kokoro-82M[12] generated samples.

LJSpeech was included because it provides high-quality, studio-recorded speech with consistent recording conditions. However, because it contains speech from a single speaker, it introduces limitations in speaker diversity. To reduce this bias, Common Voice was added to provide a wider range of speakers, accents, and recording environments. The data loading pipeline was extended to support WAV, MP3, and FLAC files, which was particularly important for handling Common Voice files consistently alongside the other datasets.

Synthetic speech samples were included from multiple generation sources to improve robustness. WaveFake provides samples generated using several neural vocoders and text-to-speech systems, including MelGAN, Parallel WaveGAN, Multi-Band MelGAN, Full-Band MelGAN[13], HiFi-GAN[14], and WaveGlow[15]. ASVspoof2019 LA provides a benchmark dataset for logical access spoofing attacks, while Kokoro-82M was used to generate additional synthetic speech samples. Including multiple fake audio sources was important because different synthesis methods introduce different artefacts. A detector trained on only one type of synthetic speech may overfit to source-specific patterns rather than learning more general indicators of artificial generation.

Type	Source	Files
Real	LJSpeech	13,100
	Common Voice	203,847
	VCTK	44,242
Real Total		261,189
Fake	Kokoro	8,419
	WaveFake	91,700
	ASVspoof 2019	122,299
Fake Total		222,418
Grand Total		483,607

Table 1: Dataset source composition showing the real speech datasets and synthetic audio sources used in the project.

To avoid the dataset being dominated by a small number of very large folders, a per-folder sampling strategy was applied. ASVspoof samples were capped at 3,000 files per subfolder, while all Kokoro samples were retained because this subset was smaller and directly generated for the project. This sampling decision helped balance the dataset and reduced the risk of the model learning dataset-source bias instead of genuine acoustic differences between real and fake speech.

All audio samples were standardised during preprocessing. Files were resampled to 16 kHz and adjusted to a fixed duration of three seconds. Shorter files were padded, while longer files were truncated. This ensured that every audio sample produced the same feature shape for model input. The dataset was then split into training, validation, and testing sets using a 70% / 15% / 15% ratio. Although the dataset contained a slight class imbalance, this was addressed through dataset balancing and class weighting during training.

Despite its size and diversity, the dataset still has limitations. LJSpeech remains a single-speaker dataset, and the fake samples are limited to known synthesis methods. Therefore, while the dataset is suitable for controlled model development and evaluation, it cannot fully represent all real-world deepfake audio generation techniques.

2.3 Feature Extraction

Feature extraction converts raw audio signals into representations that can be effectively processed by the neural network. This project uses log-Mel spectrograms as the primary feature representation because they capture both frequency and temporal characteristics of speech while remaining computationally efficient[16][17].

Each audio file is first resampled to 16 kHz and standardised to three seconds. A Short-Time Fourier Transform (STFT)[16] is then applied to convert the waveform into a time-frequency representation. The frequency axis is mapped onto the Mel scale using 128 Mel bins, reflecting the way humans perceive differences in frequency. A logarithmic transformation is then applied to compress the dynamic range and make subtle acoustic variations more visible to the model.

The feature extraction process uses an FFT window size of 2048 and a hop length of 512. For a three-second audio file, this produces a log-Mel spectrogram of shape (128, 94, 1), where 128 represents the Mel frequency bins, 94 represents the temporal frames, and the final channel dimension allows the spectrogram to be processed by convolutional layers.

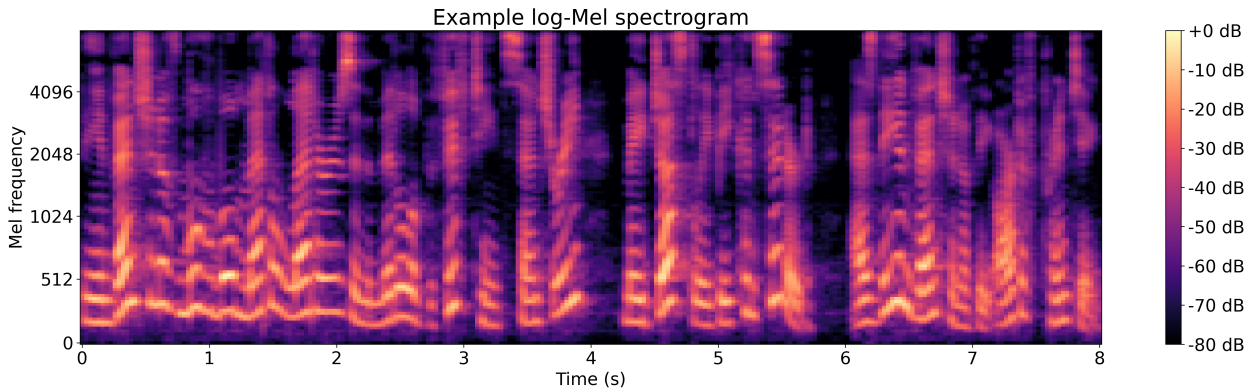


Figure 2: Example log-Mel spectrogram illustrating the time-frequency representation used for audio feature extraction.

Log-Mel spectrograms were selected because synthetic speech often contains artefacts that are more visible in time-frequency space than in the raw waveform[6][17]. These may include unnatural spectral smoothness, inconsistent harmonic structure, or temporal irregularities introduced by neural vocoders. Compared with MFCCs, log-Mel spectrograms retain more fine-grained spectral detail,

which may be useful for distinguishing high-quality synthetic speech. Compared with raw waveform models, they require less data and lower computational complexity.

Feature normalisation was applied to ensure a consistent input distribution across the dataset. A key implementation decision was to save the training-set normalisation statistics into a separate file, `norm_stats.npy`. These saved statistics are reused during inference so that new audio files are normalised in the same way as the training data. This improved consistency between training and deployment and prevented inference-time performance issues caused by mismatched normalisation.

2.4 Model Design

The classification model uses a hybrid CNN-BiLSTM architecture.[18][19][20] This design combines convolutional layers for local spectrogram feature extraction with recurrent layers for temporal sequence modelling.

The input to the model is a log-Mel spectrogram with shape $(128, 94, 1)$. The first stage consists of three convolutional blocks. Each block applies convolutional filters to learn local time-frequency patterns, followed by Batch Normalisation[21], ReLU activation, max pooling, and Dropout[22]. Batch Normalisation was included to stabilise training and reduce internal feature distribution shifts, while Dropout was included to reduce overfitting. A Dropout rate of 0.25 was applied after each convolutional block.

After the convolutional feature extraction stage, the feature maps are reshaped and passed into two Bidirectional LSTM layers. These layers model temporal dependencies in both forward and backward directions, allowing the model to consider speech patterns across the audio sequence rather than relying only on local spectrogram features. A Dropout rate of 0.3 was applied after the recurrent stage to further improve generalisation.

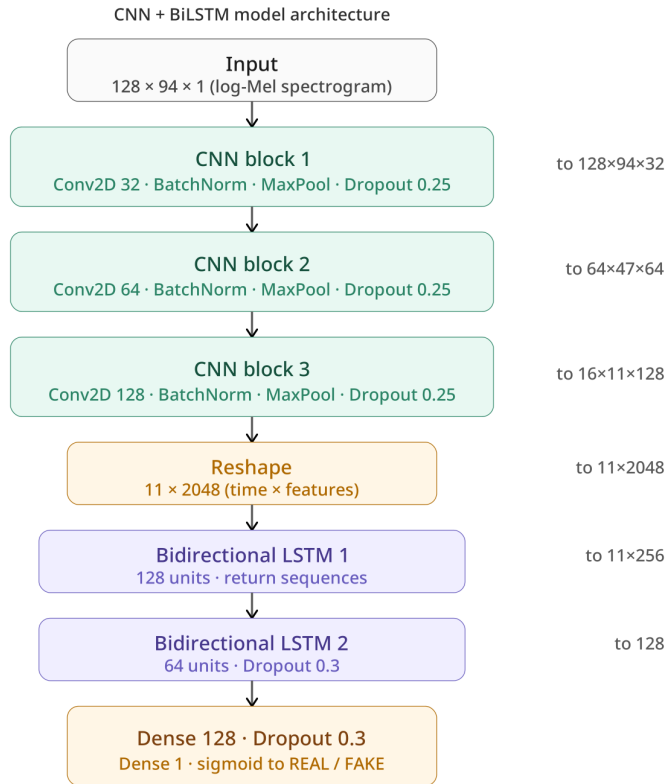


Figure 3: Architecture of the CNN-BiLSTM classifier used for deepfake audio detection.

The final classification stage consists of dense layers that map the learned representation to a

binary output. A sigmoid activation function is used to produce a probability score between 0 and 1, where higher values indicate a greater likelihood that the input audio is fake. The model contains approximately 2.5 million trainable parameters, providing sufficient capacity to learn complex acoustic patterns while remaining efficient enough for prototype inference.

The CNN-BiLSTM architecture was chosen because deepfake audio detection requires both spectral and temporal analysis. CNN layers are effective at identifying local artefacts in spectrograms, while BiLSTM layers are better suited to modelling longer-term speech dynamics. A purely CNN-based model may overlook sequential inconsistencies, while a purely recurrent model may be less effective at extracting local frequency features. Transformer-based models were considered but were not selected due to their higher computational cost and larger data requirements, which made them less suitable for the near-real-time objective of this project.

2.5 Training Strategy

The model was trained using supervised binary classification. Binary cross-entropy was used as the loss function, as it is appropriate for distinguishing between two classes: real and fake audio. The Adam optimiser was used with an initial learning rate of 1×10^{-4} [23], and the model was trained using a batch size of 64.

During early training experiments, the model showed signs of class imbalance and prediction bias. In particular, recall for genuine audio was initially low, indicating that the model was more likely to classify samples as fake. This was problematic because a practical detector must avoid excessive false alarms while still detecting synthetic speech reliably. To address this issue, class weighting was introduced so that misclassification of underrepresented or more difficult samples had a stronger effect during optimisation. The dataset was also expanded and balanced through the inclusion of additional real and synthetic sources.

Several callbacks were used to improve training stability and prevent overfitting. Early stopping was applied to stop training when validation performance no longer improved, preventing unnecessary additional epochs. Model checkpointing was used to save the best-performing version of the model during training. In addition, a `ReduceLROnPlateau` callback was used to reduce the learning rate when validation loss plateaued. During training, this callback reduced the learning rate at epoch 5, helping the model continue improving with smaller optimisation steps.

Parameter	Value
Input representation	Log-Mel spectrogram
Input shape	(128, 94, 1)
Sampling rate	16 kHz
Audio duration	3 seconds
Optimiser	Adam
Loss function	Binary cross-entropy
Learning rate	1×10^{-4}
Batch size	64
Callbacks	EarlyStopping, ModelCheckpoint, ReduceLROnPlateau
Hardware	RTX 5060 GPU via WSL2 and CUDA

Table 2: Training configuration used for the CNN-BiLSTM detector.

Training was conducted on a Windows machine using WSL2 with CUDA acceleration and an NVIDIA RTX 5060 GPU. This setup enabled faster training than CPU-only execution and made it feasible to train the CNN-BiLSTM model on the full dataset. Training stopped at epoch 16 when validation performance stopped improving, indicating that the model had reached a suitable balance between learning and generalisation.

2.6 Prototype System

To demonstrate practical applicability, a prototype inference system was implemented around the trained model. The prototype includes a prediction function, `predict_single()`, which accepts arbitrary audio files and classifies them as either *REAL* or *FAKE*. The function supports WAV, MP3, and FLAC formats, making the prototype more flexible for real-world audio inputs.

The input audio is processed using the same preprocessing and feature extraction pipeline used during training. It is resampled to 16 kHz, adjusted to three seconds, converted into a log-Mel spectrogram, and normalised using the saved `norm_stats.npy` statistics. This ensures that inference inputs are treated consistently with the data used to train the model. The trained model then outputs a probability score, which is converted into a classification label and confidence value.

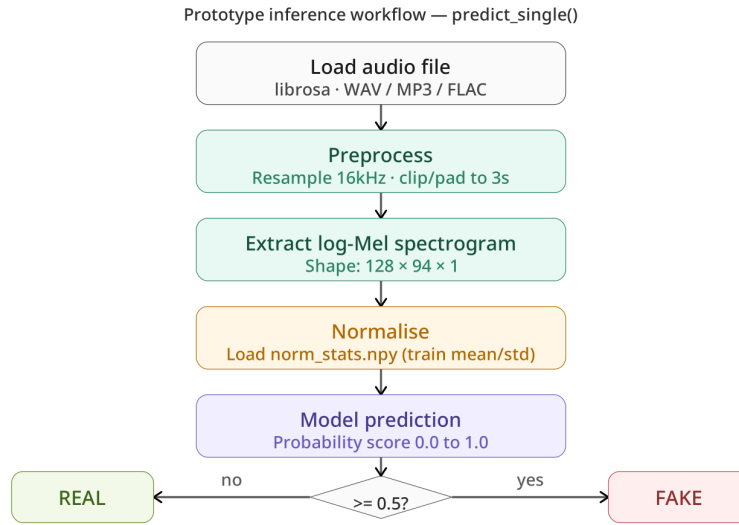


Figure 4: Prototype inference workflow for classifying unseen audio files.

The prototype was also tested on externally generated audio, including TTSMaker samples. This sample was predicted as *REAL*, despite being synthetic. This result is important because it demonstrates a key limitation of the system: although the model performs strongly on the held-out test set, it may not always generalise to unseen synthesis tools that differ from those represented in the training data[6][10]. This highlights the need for broader evaluation on external deepfake audio sources before the system could be considered reliable for real-world deployment.

Inference-time testing showed that the system can process individual files quickly, with prediction times suitable for near-real-time use. This supports the feasibility of integrating the detector into a user-facing workflow, such as screening incoming voice messages or analysing short audio segments during communication. However, the current prototype remains a proof of concept rather than a production-ready tool. Further work would be required to improve robustness, evaluate live audio streaming, and test the system against a wider range of unseen voice cloning and text-to-speech systems.

Overall, the prototype demonstrates how a trained deepfake audio detector could be incorporated into an anti-phishing workflow. It also reveals the main challenge facing practical deployment: strong internal test performance does not guarantee reliable generalisation to new synthetic audio sources.

3 Implementation

3.1 Development Environment

The system was implemented in Python using TensorFlow/Keras as the main deep learning framework. TensorFlow/Keras was used to define, train, save, and reload the CNN-BiLSTM classifier[24][25]. Supporting libraries included Librosa for audio loading and feature extraction, NumPy for numerical operations[16], scikit-learn for dataset splitting and evaluation utilities[26], and Matplotlib for generating visualisations.

Development and training were conducted on a Windows machine using WSL2 with CUDA acceleration and an NVIDIA RTX 5060 GPU. GPU support was important because the project involved processing a large number of audio files and training a neural network on log-Mel spectrogram features. The final model was saved as `deepfake_detector.keras`, while the training-set normalisation statistics were saved separately as `norm_stats.npy`. These saved artefacts allowed the trained model and its preprocessing assumptions to be reused during prototype inference.

The implementation was developed as an iterative notebook-based pipeline. The main stages included dataset scanning, audio preprocessing, feature extraction, model training, evaluation, and single-file inference. This structure allowed each part of the system to be tested and improved independently before being combined into the final pipeline.

3.2 Data Loading and Preprocessing

The data loading pipeline was designed to collect audio samples from multiple real and synthetic sources. In the final version, genuine audio was sourced from LJSpeech, VCTK, and Common Voice, while synthetic audio was sourced from Kokoro-82M, WaveFake, and ASVspoof2019 Logical Access (LA). Each audio file was assigned a binary label according to its source, with genuine recordings labelled as *REAL* and synthetic or spoofed samples labelled as *FAKE*.

A key implementation requirement was support for multiple audio formats. Earlier versions only supported WAV files, but the final pipeline was extended to support WAV, MP3, and FLAC formats. This was necessary because the datasets used different file formats: Common Voice commonly uses MP3 files, while ASVspoof2019 includes FLAC files. Support for these formats was implemented through Librosa with ffmpeg and audioread backend support.

Each audio file was resampled to 16 kHz and standardised to a fixed duration of three seconds. Audio shorter than three seconds was padded, while longer audio was truncated. This ensured that every sample produced a consistent input size for feature extraction and model training. The final dataset was divided into training, validation, and testing sets using a stratified 70% / 15% / 15% split, ensuring that both real and fake classes were represented across all subsets.

A sampling strategy was also implemented to reduce dataset-source imbalance. ASVspoof2019 was capped at 3,000 files per subfolder to prevent it from dominating the fake class. Kokoro-82M samples were retained in full because they represented the project’s own generated synthetic audio. Remaining fake sources were sampled more evenly across vocoder subfolders, while the real class was randomly sampled to maintain a more balanced final dataset.

3.3 Feature Extraction and Normalisation

Feature extraction was performed during runtime rather than being cached to disk. For each audio file, the pipeline loaded the raw waveform, resampled it, adjusted it to the required three-second duration, and converted it into a log-Mel spectrogram using Librosa.

The feature extraction process used 128 Mel bins, an FFT window size of 2048, and a hop length of 512. Each processed file produced a log-Mel spectrogram with shape (128, 94, 1), where 128 represents the Mel frequency bins, 94 represents the temporal frames, and the final channel dimension allows the spectrogram to be used as input to convolutional layers.

A significant implementation issue identified during development was inconsistent normalisation between training and inference. In the first version, the `predict_single()` function normalised each input file using its own mean and standard deviation. This meant that each inference sample was scaled independently, creating a distribution mismatch between training and prediction.

To resolve this, the final implementation calculates the mean and standard deviation from the training data and saves them to `norm_stats.npy`. During inference, `predict_single()` loads these saved statistics and applies the same normalisation process used during training. This ensures that new audio files are processed consistently with the model’s training distribution.

3.4 Model Implementation

The classifier was implemented as a CNN-BiLSTM model in TensorFlow/Keras. The model accepts log-Mel spectrograms with shape (128, 94, 1) as input. The convolutional section consists of three CNN blocks, each designed to extract local time-frequency patterns from the spectrogram representation.

Each convolutional block includes convolutional processing, Batch Normalisation, ReLU activation, max pooling, and Dropout. Batch Normalisation was included to stabilise training and improve convergence, while Dropout was used to reduce overfitting. A Dropout rate of 0.25 was applied after each CNN block.

After the convolutional section, the extracted feature maps were reshaped and passed into two Bidirectional LSTM layers. These recurrent layers model temporal dependencies in the speech signal by processing the sequence in both forward and backward directions. A Dropout rate of 0.3 was applied after the recurrent stage to further improve generalisation.

The final dense layers map the learned representation to a single sigmoid output. This produces a probability score for the fake class, where values closer to 1 indicate a higher likelihood of synthetic audio and values closer to 0 indicate a higher likelihood of genuine audio. The final model contained approximately 2.5 million trainable parameters.

3.5 Training Implementation

The model was trained as a supervised binary classification system using binary cross-entropy loss. The Adam optimiser was used with an initial learning rate of 1×10^{-4} , and training was performed with a batch size of 64. Class weights were applied in the final training run to address the remaining imbalance between real and fake samples.

Several callbacks were used to improve training stability. `ModelCheckpoint` saved the best-performing model during training, `EarlyStopping` stopped training when validation performance no longer improved, and `ReduceLROnPlateau` reduced the learning rate when validation performance plateaued. In the final run, the learning rate was reduced at epoch 5 from 1×10^{-4} to 5×10^{-5} , and training stopped at epoch 16.

The final model was selected from the best saved checkpoint rather than simply using the last epoch. This helped ensure that the final deployed model represented the strongest validation performance observed during training.

3.6 Prototype Inference Implementation

A prototype inference function, `predict_single()`, was implemented to test the trained model on arbitrary audio files. The function accepts an audio file path as input and returns a predicted label, fake probability, and real/fake confidence values. It supports WAV, MP3, and FLAC files, making it more flexible than the earlier WAV-only version.

The inference pipeline mirrors the training pipeline. The input audio is loaded, resampled to 16 kHz, adjusted to three seconds, converted into a log-Mel spectrogram, and normalised using the saved `norm_stats.npy` values. The processed feature tensor is then passed to the trained Keras model. The final prototype uses a default decision threshold of 0.5: outputs greater than or equal to 0.5 are classified as *FAKE*, while outputs below 0.5 are classified as *REAL*.

Threshold tuning was later performed for evaluation and analysis, but the tuned threshold was not applied to the deployed `predict_single()` function. This distinction is important because the prototype behaviour reflects the implemented system, while threshold tuning is discussed separately as part of the evaluation.

The prototype was tested on external TTSMaker samples. Although the sample was synthetic, the model predicted it as *REAL*, with 99.82% real confidence and only 0.18% fake confidence. This result highlighted an important limitation: strong performance on the internal held-out test set does not guarantee reliable generalisation to every unseen synthesis tool.

3.7 Iterative Development

The final implementation was developed through three main versions. This iterative process was important because each version revealed practical problems that informed the next stage of development.

3.7.1 Version 1: Initial Implementation

The first version established the core pipeline using an early CNN-BiLSTM architecture. This design already contained the main structure retained in later versions: log-Mel spectrogram extraction, three convolutional blocks, two Bidirectional LSTM layers, and a dense binary classification head trained with binary cross-entropy loss.

However, Version 1 had several limitations. It only supported WAV files, sampled fake files randomly without per-source balancing, and used per-file normalisation during single-file inference. The dataset was also severely imbalanced, containing approximately 10,000 real samples and 110,000 fake samples. Although class weights were applied during training, the imbalance still strongly affected model behaviour.

The model achieved 93.75% accuracy, but this result was misleading. Real precision was 1.00, but real recall was only 0.30, meaning that around 70% of genuine samples were incorrectly classified as fake. Fake precision was 0.94 and fake recall was 1.00. This showed that the model had learned to predict *FAKE* as a default strategy rather than learning a balanced distinction between real and synthetic speech. Such behaviour would be unsuitable for a practical anti-phishing detector, as excessive false alarms would reduce user trust.

3.7.2 Version 2: Normalisation Fix and Dataset Expansion

Version 2 addressed the most important problems identified in Version 1. The first improvement was fixing the inference normalisation process. Instead of normalising each inference file independently, `predict_single()` was updated to load the saved training-set statistics from `norm_stats.npy`. This ensured that single-file inference used the same scaling assumptions as training.

The second improvement was dataset expansion. VCTK was added to the real class to increase speaker diversity beyond LJSpeech, while Kokoro-82M was added as a synthetic source. This increased the number of genuine samples and reduced the real-to-fake imbalance compared with Version 1.

Although Version 2 improved recognition of genuine audio, it revealed a new issue. The model still showed signs of overfitting and dataset-source bias. The Version 2 confusion matrix showed 3,530 real samples correctly classified as real, but 970 real samples incorrectly classified as fake. In contrast, only 21 fake samples were misclassified as real, while 3,924 fake samples were correctly classified as fake. This indicated that the model had improved compared with Version 1, but it still produced too many false alarms on real audio.

The training curves also suggested overfitting. Training loss continued to decrease towards zero, while validation loss remained high and unstable. This indicated that the model was learning patterns specific to the training distribution rather than fully generalisable real/fake audio characteristics.

3.7.3 Version 3: Final Pipeline

Version 3 incorporated the lessons from the previous two versions and became the final implementation used for evaluation. This version added Common Voice to the real class and ASVspoof2019 LA to the fake class, while retaining the previous sources. The addition of Common Voice improved real-speaker and recording-condition diversity, while ASVspoof2019 added benchmark spoofing data to the fake class.

To prevent ASVspoof2019 from dominating the fake dataset, a per-folder sampling cap of 3,000 files was applied. Kokoro-82M samples were retained in full, and remaining fake sources were sampled more evenly across vocoder subfolders. Multi-format support was also added so that WAV, MP3, and FLAC files could all be processed consistently.

The final training configuration used a stratified 70% / 15% / 15% train-validation-test split, class weights, a batch size of 64, and an initial learning rate of 1×10^{-4} . The learning rate was reduced at epoch 5, and training stopped at epoch 16. The final model achieved a substantially more balanced error profile than Version 2, reducing real samples incorrectly flagged as fake from 970 in Version 2 to 84 in Version 3.

However, Version 3 also introduced a trade-off. The number of fake samples incorrectly classified as real increased from 21 in Version 2 to 150 in Version 3. This shows that the final system became much less aggressive in labelling audio as fake, which reduced false alarms but slightly increased missed synthetic samples. This trade-off is important in a phishing defence context because both false alarms and missed attacks affect practical usability.

3.8 Implementation Challenges

Several implementation challenges were encountered during development. The first challenge was class imbalance. Version 1 showed that high overall accuracy could hide poor real-audio recall, making the model unsuitable for practical use. This was addressed through dataset expansion, class weighting, and more balanced sampling.

The second challenge was dataset-source bias. Version 2 showed that adding more data was not sufficient if one dataset source dominated a class. The final sampling strategy in Version 3 was introduced to reduce this risk and encourage the model to learn broader real/fake characteristics rather than source-specific artefacts.

The third challenge was consistency between training and inference. The initial per-file normalisation approach caused inference samples to be processed differently from the training data. Saving and reusing `norm_stats.npy` resolved this issue and made the prototype inference pipeline more reliable.

The fourth challenge was generalisation to unseen synthesis tools. The TTSMaker test showed that the final model could still fail on synthetic audio generated by tools not represented in the training dataset. This limitation does not invalidate the implementation, but it demonstrates that broader external testing is required before the system could be considered reliable for real-world deployment.

Overall, the implementation evolved from a basic but biased classifier into a more complete deepfake audio detection pipeline. The final version includes multi-source data loading, multi-format audio support, consistent feature extraction and normalisation, a regularised CNN-BiLSTM model, class-weighted training, and a prototype inference function suitable for testing unseen audio files.

4 Results and Evaluation

4.1 Evaluation Overview

The final CNN-BiLSTM model was evaluated using two forms of analysis. First, the main classification results were calculated using the full held-out v3 test set of 13,532 samples. This full test set is used for the official accuracy, precision, recall, F1-score, and confusion matrix analysis. Second,

additional visualisations, including the ROC curve, precision-recall curve, threshold tuning plot, per-class performance chart, and inference-time analysis, were generated using a balanced 2,000-sample evaluation subset. This subset contained 1,000 real and 1,000 fake samples and was used to support interpretation because the exact saved file list for the full test split was not retained.

This distinction is important for transparency. The full held-out test set provides the main evidence of model performance, while the 2,000-sample subset provides additional visual insight into model behaviour. The subset visualisations should therefore be interpreted as supporting analysis rather than as replacements for the full test-set metrics.

In the context of deepfake phishing detection, the *FAKE* class is particularly important because it represents synthetic audio that could be used for impersonation or voice phishing. A false negative, where fake audio is classified as real, is security-critical because it may allow a malicious audio sample to pass undetected. However, false positives are also important because incorrectly flagging genuine audio as fake could reduce user trust and make the system impractical. For this reason, the evaluation considers both fake detection performance and the balance between missed attacks and false alarms.

4.2 Full Test Set Performance

The final v3 model achieved an overall accuracy of 98.27% on the full held-out test set of 13,532 samples. Table 3 summarises the class-specific results.

Class	Precision	Recall	F1-score	Support
Real	0.9802	0.9888	0.9845	7500
Fake	0.9859	0.9751	0.9805	6032
Accuracy	98.27%			

Table 3: Classification performance of the final v3 model on the full held-out test set.

The results show strong performance for both real and fake audio. The real class achieved a recall of 0.9888, meaning that most genuine samples were correctly identified. This is important for usability because a practical anti-phishing tool should not frequently raise false alarms during normal communication. The fake class achieved a recall of 0.9751, showing that the model detected the majority of synthetic samples. However, this also means that some fake samples were missed, which remains a concern in a security-sensitive setting.

The fake class achieved slightly higher precision than the real class, while the real class achieved slightly higher recall. This indicates that the model was slightly cautious when assigning the fake label: when it predicted fake, it was usually correct, but a small number of fake samples were still classified as real.

4.3 Confusion Matrix Analysis

Figure 5 shows the confusion matrix for the final v3 model on the full test set.

The model correctly classified 7,416 genuine samples and 5,882 fake samples. Misclassifications were limited to 84 genuine samples incorrectly classified as fake and 150 fake samples incorrectly classified as real.

These results show a much more balanced error profile than earlier development versions. In Version 2, 970 real samples were incorrectly classified as fake, while in the final Version 3 model this number was reduced to 84. This suggests that the final dataset balancing strategy and preprocessing improvements substantially improved the model’s ability to recognise genuine audio.

However, this improvement also introduced a trade-off. Version 2 missed only 21 fake samples, whereas Version 3 missed 150 fake samples. This means that the final model became less aggressive in labelling audio as fake. From a practical perspective, this improves usability by reducing false alarms, but it also increases the number of synthetic samples classified as real. In a phishing defence

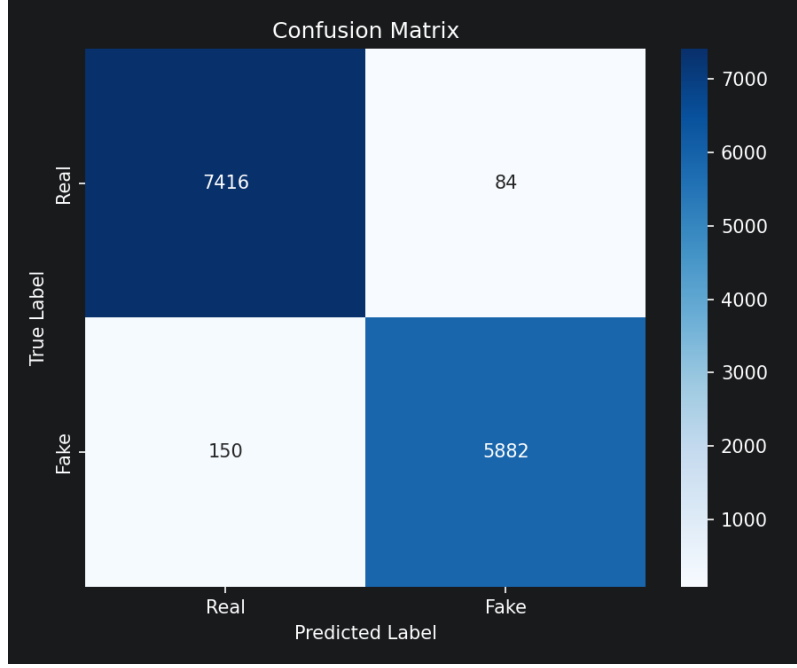


Figure 5: Confusion matrix for the final CNN-BiLSTM model on the full held-out test set.

context, this trade-off is important because both excessive false alarms and missed attacks can reduce the effectiveness of the system.

4.4 Visual Evaluation on Balanced Subset

Additional visual analysis was performed using a balanced 2,000-sample evaluation subset. This subset was used to generate clearer visualisations of model behaviour across different thresholds and classes. Since these figures were not generated from the exact full 13,532-sample test set, they are interpreted as supplementary analysis rather than the official performance results.

4.4.1 ROC Curve

Figure 6 presents the ROC curve generated from the balanced evaluation subset.

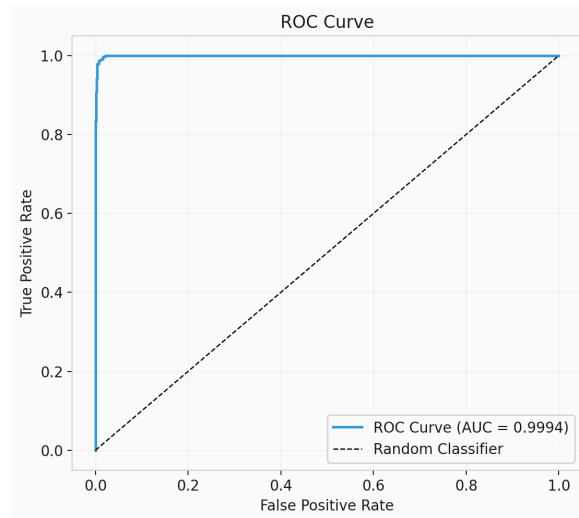


Figure 6: ROC curve generated from the balanced 2,000-sample evaluation subset.

The ROC curve shows very strong separability between real and fake samples within the known-source evaluation subset. This indicates that the model can rank fake samples above real samples effectively under controlled internal evaluation conditions.

However, this result should not be interpreted as proof of universal deepfake detection. The subset was sampled from the same overall dataset sources used during development, so the model may partly benefit from recognising source-specific or generator-specific artefacts. Therefore, the ROC curve demonstrates strong internal discrimination, but not guaranteed generalisation to unseen synthesis tools.

4.4.2 Precision-Recall Curve

Figure 7 shows the precision-recall curve for the same balanced subset.

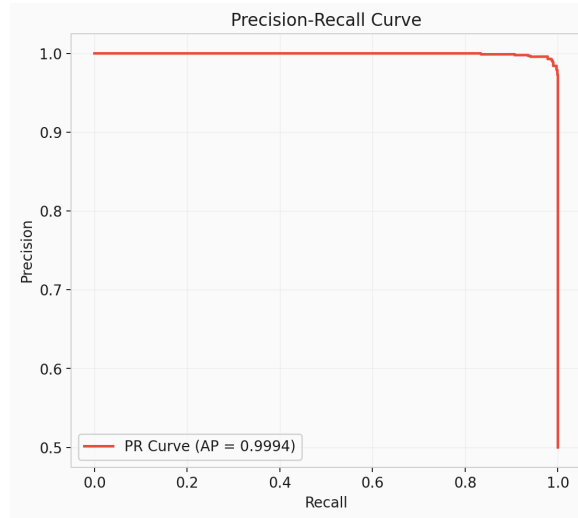


Figure 7: Precision-recall curve generated from the balanced 2,000-sample evaluation subset.

The precision-recall curve demonstrates that the model maintains high precision while preserving high recall across a range of thresholds on the subset. This supports the conclusion that the model performs strongly on known-source real and fake audio. In a phishing detection system, this is useful because the decision threshold could potentially be adjusted depending on whether the deployment context prioritises sensitivity or reduced false alarms.

Nevertheless, as with the ROC curve, the precision-recall curve should be interpreted as internal evaluation evidence. It does not remove the need for testing on external and unseen text-to-speech systems.

4.5 Threshold Tuning

Threshold tuning was performed on the 2,000-sample subset to examine how classification behaviour changes when the decision threshold is adjusted. The implemented `predict_single()` prototype uses the default threshold of 0.5, meaning that probability scores greater than or equal to 0.5 are classified as *FAKE*, while scores below 0.5 are classified as *REAL*. The threshold tuning analysis was performed only for evaluation and was not applied to the deployed prototype.

The threshold tuning results suggest that the best F1-score on the subset occurs at a lower threshold than 0.5. This indicates that the model’s probability outputs are not perfectly calibrated around the default decision boundary. In a high-security scenario, lowering the threshold could increase fake recall and reduce the risk of missing synthetic audio. However, doing so would likely increase the number of genuine samples incorrectly flagged as fake.

For this project, the threshold tuning result is treated as an analytical finding rather than an implementation change. The prototype retained the default threshold of 0.5 to keep the deployed

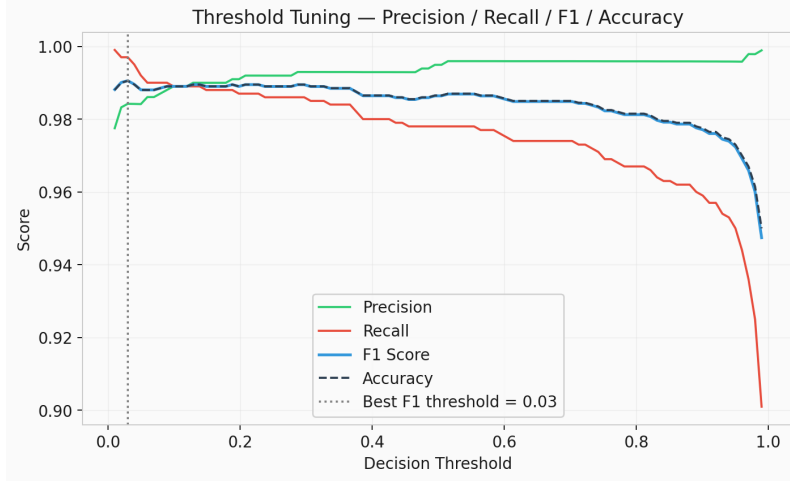


Figure 8: Threshold tuning analysis on the balanced 2,000-sample evaluation subset.

inference logic simple and transparent. Future work could apply probability calibration and select thresholds according to the risk level of the deployment environment.

4.6 Per-Class Performance on the Subset

Per-class performance was also visualised using the balanced evaluation subset, as shown in Figure 9.



Figure 9: Per-class precision, recall, and F1-score on the balanced 2,000-sample evaluation subset.

The per-class visualisation shows that performance remains high for both real and fake audio within the subset. This supports the full test-set result that the model does not show extreme bias toward one class. However, the subset result should be considered alongside the full confusion matrix, which provides the more reliable measure of final performance.

4.7 Dataset and Source Analysis

Dataset composition was analysed because earlier development versions showed that class imbalance and source imbalance could produce misleadingly strong accuracy. In particular, Version 1 achieved high overall accuracy while performing poorly on genuine audio, and Version 2 still produced too

many false alarms on real samples. This demonstrated that dataset structure directly affected model behaviour.

The final dataset included multiple real sources, including LJSpeech, VCTK, and Common Voice, and multiple fake sources, including Kokoro-82M, WaveFake, and ASVspoof2019 LA. This improved diversity compared with earlier versions. The fake-source breakdown is particularly important because different synthetic speech systems can introduce different artefacts. Training on multiple fake sources reduces the risk of the model learning only one generator’s characteristics.

However, the dataset still cannot cover all possible synthesis tools. The model can only learn from the real and fake distributions represented during training. As a result, strong internal performance does not guarantee that the detector will generalise to every unseen voice cloning or text-to-speech system.

4.8 Inference Time Evaluation

Inference time was evaluated to assess whether the system could support near-real-time detection of short audio samples. Figure 10 presents the inference-time analysis.

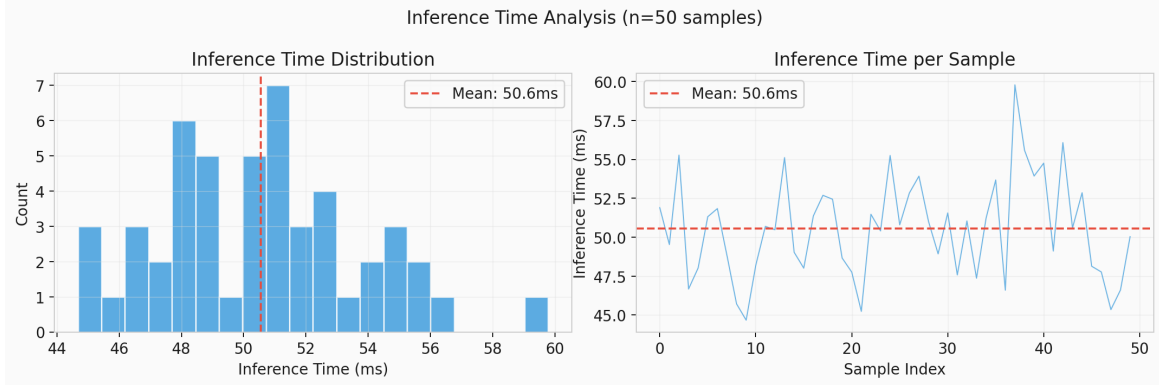


Figure 10: Prototype inference-time analysis for single-file prediction.

The results show that the system can process short audio files efficiently, with an average prediction time of approximately 50.6 ms per file in the tested setup. This suggests that the model is computationally practical for near-real-time analysis of short audio clips.

However, this timing should be interpreted as prototype-level evidence rather than production latency. The current implementation performs single-file inference, not continuous live-call analysis. A deployed real-time system would require additional components such as audio buffering, overlapping window processing, latency management, and integration with communication software. Therefore, the inference-time result supports feasibility but does not by itself prove deployment readiness.

4.9 External Generalisation Test

To evaluate behaviour outside the internal dataset, the prototype was tested on five external synthetic audio samples generated using TTSMaker. These samples were not part of the training or test data and therefore represent an unseen text-to-speech source.

Despite all samples being synthetic, the model classified every sample as *REAL*. The predicted fake confidence scores ranged from 0.00% to 5.78%, while real confidence remained consistently above 94%.

This result is a critical finding. It demonstrates that although the model performs well on held-out data from known datasets, it does not generalise reliably to unseen synthesis systems. The consistently high confidence in incorrect predictions suggests that the model has learned source-specific artefacts rather than universal characteristics of synthetic speech.

This limitation is particularly significant in the context of phishing detection. In real-world scenarios, attackers are likely to use a wide variety of evolving text-to-speech tools. A detection system that fails on unseen generators would therefore be ineffective in practice. Consequently, while the proposed system is a strong proof of concept, it is not yet suitable for real-world deployment.

Future work should address this limitation by incorporating a broader range of modern TTS systems into the training data and evaluating performance on multiple external datasets to improve generalisation.

4.10 Overall Evaluation

Overall, the final model demonstrates strong internal performance. On the full held-out v3 test set, it achieved 98.27% accuracy, with high precision, recall, and F1-scores for both real and fake audio. The final confusion matrix also shows a major improvement over earlier development versions, particularly in reducing the number of genuine samples incorrectly flagged as fake.

The visual subset analysis supports this conclusion by showing strong ROC, precision-recall, threshold, and per-class behaviour under controlled evaluation conditions. The inference-time result also suggests that the system is efficient enough for near-real-time analysis of short audio segments.

However, the evaluation also identifies clear limitations. The model missed 150 fake samples in the full test set, and the external TTSMaker sample was confidently misclassified as real. This means the system should be understood as a strong proof of concept rather than a fully generalisable deepfake audio detector. Its main contribution is the development and evaluation of a complete deepfake audio detection pipeline for anti-phishing scenarios, while its main limitation is robustness against unseen synthesis models.

5 Discussion and Reflection

5.1 Discussion

The results of this project show that machine learning can be effective for detecting deepfake audio under controlled evaluation conditions. The final CNN-BiLSTM model achieved strong internal performance on the held-out test set, with high accuracy, precision, recall, and F1-scores across both real and fake classes. This suggests that the combination of log-Mel spectrogram features and a CNN-BiLSTM architecture is suitable for learning meaningful differences between genuine and synthetic speech.

The use of log-Mel spectrograms was an important design choice because they represent audio across both time and frequency dimensions. This allowed the CNN layers to capture local spectral artefacts[27], while the BiLSTM layers modelled temporal patterns across speech. The results indicate that this hybrid architecture was appropriate for the task, as deepfake audio detection depends on both local acoustic features and sequential speech dynamics.

A major finding from the development process was that high accuracy alone can be misleading. The first implementation achieved high overall accuracy, but real recall was poor, meaning that many genuine samples were incorrectly classified as fake. This showed that the model had learned a biased decision strategy due to dataset imbalance. The final version improved this through dataset expansion, class weighting, multi-format support, saved normalisation statistics, and balanced sampling. This demonstrates that dataset design and preprocessing were as important as the model architecture itself.

The final confusion matrix also shows a practical trade-off. The final model greatly reduced the number of genuine samples incorrectly flagged as fake, improving usability. However, it still missed some fake samples. In an anti-phishing context, this trade-off is important. A system that produces too many false alarms may be ignored by users, but a system that misses synthetic speech may fail to prevent an attack. Therefore, the best threshold and operating mode would depend on the deployment context.

5.2 Limitations

The most significant limitation is generalisation. Although the model performed strongly on the internal held-out test set, the external TTSMaker test showed that it does not reliably generalise to all unseen synthesis tools. Five additional synthetic TTSMaker samples were tested, and all five were classified as *REAL*. Fake confidence remained very low, ranging from 0.00% to 5.78%, while real confidence stayed above 94%. This corresponds to a 0% detection rate on this small external set.

This result suggests that the model may have learned artefacts associated with the fake sources included in the training data, rather than universal indicators of synthetic speech. This limitation is highly relevant to phishing detection because real attackers may use tools that are not represented in the training dataset. Therefore, the system should be interpreted as a controlled proof of concept rather than a deployment-ready universal detector.

A second limitation is that the prototype uses single-file inference rather than live streaming analysis. Although inference time was low for short audio files, a real-time communication system would require additional engineering, including audio buffering, sliding-window analysis, latency control, and integration with calling or messaging software.

A third limitation is that the system does not verify speaker identity. It classifies whether audio appears real or fake, but it does not confirm whether the speaker is actually the person they claim to be. In practical anti-phishing systems, deepfake detection would likely need to be combined with identity verification, caller authentication, or organisational verification workflows.

Finally, the system currently focuses only on audio. This makes the project more manageable and relevant to voice phishing, but it limits effectiveness against multimodal attacks involving both cloned audio and manipulated video.

5.3 Reflection

This project successfully developed a complete deepfake audio detection pipeline, including dataset construction, preprocessing, feature extraction, model training, evaluation, and prototype inference. The final system met the core aim of building an AI-based detector capable of classifying audio as real or fake within a controlled evaluation setting.

One strength of the project is that it evolved through an iterative development process rather than relying on a single implementation. Earlier versions revealed problems such as class imbalance, unstable real-class recall, and inference normalisation mismatch. These issues were addressed through dataset expansion, class weighting, balanced sampling, multi-format support, and the saving and reuse of `norm_stats.npy`. This process provided clear evidence of problem identification, correction, and refinement.

Another strength is that the project went beyond using a single dataset. The final dataset included multiple real and synthetic sources, including LJSpeech, Common Voice, WaveFake, ASVspoof2019 LA, and Kokoro-82M generated audio. This helped reduce dependence on one source and made the evaluation more meaningful than a single-dataset experiment.

A further reflection concerns the project scope. The original proposal considered a broader audio-video deepfake phishing system. During development, the scope was narrowed to audio-only detection due to practical constraints. A multimodal system would require video datasets, frame-level preprocessing, lip-sync analysis, and audio-visual synchronisation, all of which would significantly increase project complexity. However, the audio-only direction remains relevant because many voice phishing and phone-based scams rely entirely on speech, with no visual evidence available to the victim.

Overall, the project was successful as a proof of concept, but not as a complete production-ready security tool. Its main value is demonstrating the feasibility of a deepfake audio detection pipeline while clearly identifying the practical barriers that remain.

5.4 Evaluation Against Objectives

Table 4: Evaluation against project objectives

ID	Objective	Outcome
O1	Construct a dataset containing genuine and synthetic audio samples for deep-fake detection.	Achieved. A large multi-source dataset was constructed using LJSpeech, VCTK, Common Voice, WaveFake, ASVspoof2019 LA, and Kokoro-82M generated audio.
O2	Develop an audio preprocessing and feature extraction pipeline suitable for machine learning.	Achieved. Audio was standardised to 16 kHz, converted into fixed-length three-second samples, and represented using log-Mel spectrograms. Normalisation statistics were saved and reused during inference.
O3	Design and train a model capable of distinguishing between genuine and synthetic audio.	Achieved. A CNN-BiLSTM model was developed and trained using binary cross-entropy, class weighting, dropout, batch normalisation, early stopping, and learning rate scheduling.
O4	Evaluate the model using appropriate performance metrics and analysis.	Achieved. The model was evaluated using accuracy, precision, recall, F1-score, confusion matrix analysis, ROC curve, precision-recall curve, threshold tuning, and per-class performance analysis.
O5	Develop a prototype inference workflow for analysing unseen audio files.	Achieved as a proof of concept. The <code>predict_single()</code> function can classify WAV, MP3, and FLAC files and return confidence scores. However, external TTSMaker testing showed poor generalisation to unseen synthesis tools, so the prototype is not deployment-ready.

5.5 Future Work

Future work should first focus on improving cross-generator generalisation. The dataset should be expanded to include a wider range of modern text-to-speech and voice cloning systems, including tools not represented in the current training data. External test sets should also be separated by generator, so that the model can be evaluated on completely unseen synthesis methods rather than only random samples from known sources.

A second direction is threshold calibration. The current prototype uses a default threshold of 0.5, while threshold tuning was only used for analysis. Future work could apply probability calibration and select thresholds based on deployment risk. For example, a high-security mode could prioritise fake recall, while a general screening mode could prioritise fewer false alarms.

A third improvement would be real-time streaming integration. Instead of classifying pre-recorded three-second files, the system could process audio in overlapping windows during a live call. This would require buffering, latency measurement, and aggregation of predictions across multiple windows to avoid unstable alerts.

A fourth direction is multimodal detection. Although the final project focused on audio, future work could combine audio detection with visual cues such as lip-sync consistency, facial artefacts, and speaker verification. This would be especially useful for video-call phishing attacks, where attackers may combine cloned voices with synthetic or manipulated video.

Finally, future work could explore more advanced architectures, such as transformer-based audio models, self-supervised speech models, or ensemble approaches. These may improve robustness, but they should be compared carefully against the current CNN-BiLSTM model in terms of accuracy, inference time, and generalisation to unseen generators.

6 Conclusion

This project set out to develop an AI-based system for detecting deepfake audio in phishing and impersonation scenarios. The final system achieved strong internal performance, with 98.27% accuracy on the full held-out v3 test set and balanced precision and recall across real and fake audio.

The project demonstrates that log-Mel spectrograms combined with a CNN-BiLSTM architecture can effectively distinguish genuine and synthetic speech under controlled conditions. It also shows that implementation details such as dataset balancing, consistent normalisation, class weighting, and multi-format audio support have a major impact on model reliability.

However, the project also demonstrates that strong internal performance is not enough for real-world deployment. The external TTSMaker test showed that the model can fail on unseen synthesis tools, highlighting the central challenge of generalisation in deepfake detection. Therefore, the system should be understood as a successful proof of concept rather than a production-ready phishing defence tool.

Overall, the project contributes a complete deepfake audio detection pipeline and provides useful evidence for the feasibility and limitations of machine learning-based anti-phishing systems. With broader datasets, external generator testing, calibrated thresholds, and real-time integration, this work could form the basis for more robust defences against deepfake-enabled voice phishing and wider synthetic audio misuse.

References

- [1] Y. Mirsky and W. Lee, “The creation and detection of deepfakes: A survey,” *ACM Computing Surveys*, vol. 54, no. 1, pp. 1–41, 2021.
- [2] R. Chesney and D. Citron, “Deepfakes and the new disinformation war,” *Foreign Affairs*, vol. 98, no. 1, pp. 147–155, 2019.
- [3] E. Lastdrager, “Achieving a consensual definition of phishing based on a systematic review of the literature,” *Crime Science*, vol. 3, no. 1, p. 9, 2014.
- [4] CISA/NSA/FBI, “Contextualising deepfake threats to organisations,” Joint Cybersecurity Information Sheet, Sep. 2023. [Online]. Available: <https://www.cisa.gov/resources-tools/resources/contextualizing-deepfake-threats-organizations>
- [5] E. Ongweso Jr., “Thieves used audio deepfake of a CEO to steal \$243,000,” VICE News, Sep. 2019. [Online]. Available: <https://www.vice.com/en/article/thieves-deepfake-ceo-voice/>
- [6] J. Yi, C. Wang, J. Tao, X. Zhang, C. Y. Zhang, and Y. Zhao, “Audio deepfake detection: A survey,” *arXiv preprint arXiv:2308.14970*, 2023.
- [7] K. Ito and L. Johnson, “The LJ speech dataset,” <https://keithito.com/LJ-Speech-Dataset/>, 2017.
- [8] R. Ardila *et al.*, “Common voice: A massively-multilingual speech corpus,” in *Proc. 12th Language Resources and Evaluation Conference (LREC)*, 2020, pp. 4218–4222.
- [9] J. Frank and L. Schönherr, “WaveFake: A data set to facilitate audio deepfake detection,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2021.
- [10] X. Wang *et al.*, “ASVspoof 2019: A large-scale public database of synthesized, converted and replayed speech,” *Computer Speech & Language*, vol. 64, p. 101114, 2020.
- [11] M. Todisco *et al.*, “ASVspoof 2019: Future horizons in spoofed and fake audio detection,” in *Proc. INTERSPEECH 2019*, 2019, pp. 1008–1012.
- [12] hexgrad, “Kokoro-82m: A text-to-speech model,” GitHub, 2024. [Online]. Available: <https://github.com/hexgrad/kokoro>
- [13] K. Kumar *et al.*, “MelGAN: Generative adversarial networks for conditional waveform synthesis,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [14] J. Kong, J. Kim, and J. Bae, “HiFi-GAN: Generalizable end-to-end generative adversarial network for real-time high-fidelity speech synthesis,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [15] R. Prenger, R. Valle, and B. Catanzaro, “WaveGlow: A flow-based generative network for speech synthesis,” in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019.
- [16] B. McFee *et al.*, “librosa: Audio and music signal analysis in Python,” in *Proc. 14th Python in Science Conference (SciPy)*, 2015, pp. 18–24.
- [17] L. Pham, P. Lam, T. Nguyen, H. Nguyen, and A. Schindler, “Deepfake audio detection using spectrogram-based feature and ensemble of deep learning models,” *arXiv preprint arXiv:2407.01777*, 2024.

- [18] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [19] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [20] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [21] S. Ioffe and C. Szegedy, “Batch normalisation: Accelerating deep network training by reducing internal covariate shift,” in *Proc. 32nd International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.
- [22] N. Srivastava *et al.*, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [23] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimisation,” *arXiv preprint arXiv:1412.6980*, 2015.
- [24] M. Abadi *et al.*, “TensorFlow: Large-scale machine learning on heterogeneous systems,” <https://www.tensorflow.org>, 2015.
- [25] F. Chollet *et al.*, “Keras,” GitHub, 2015. [Online]. Available: <https://github.com/keras-team/keras>
- [26] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [27] T. P. Doan *et al.*, “BTS-E: Audio deepfake detection via breathing–talking synchronisation,” in *Proc. INTERSPEECH 2023*, 2023.

Generative AI Declaration

I declare that for this assessment, the following applies regarding the use of Generative AI tools:

- ☐ I have used GenAI tools to develop ideas.
- ☐ I have used GenAI tools to assist with research or information gathering.
- ☒ I have used GenAI tools to help me understand key theories and concepts.
- ☐ I have used GenAI tools to identify trends and themes as part of data analysis.
- ☒ I have used GenAI tools to provide feedback on a draft
- ☒ I have used GenAI tools to improve a plan or structure for my assessment.
- ☐ I have used GenAI tools to identify trends and themes as part of data analysis
- ☒ I have used GenAI tools to generate images, figures or diagrams.
- ☒ I have used GenAI tools to proofread and correct grammar or spelling errors.
- ☐ I have used GenAI tools to generate citations or references.
- ☐ I have used GenAI tools to translate material into English.

A Appendix

A.1 Generative AI Prompt Log

- ChatGPT prompt used for clear concepts:
<https://chatgpt.com/share/69fad057-dfb8-8391-b4dd-0bc0c000ac02>
- ChatGPT prompt used for improve project plan:
<https://chatgpt.com/share/69fad0f3-97c0-838d-a821-7046bc178409>
- Claude prompt used for generate image:
<https://claude.ai/share/b506276d-f99a-4134-af37-df2fe82f2110>